Software Engineering Lab

Lab 32

Database Sharding & Table Partitioning

Logical Pruning vs Physical Sharding, Consistent Hashing, dan Eliminasi Scatter-Gather



Lukman / lukman-ss

Masalah Nyata: Batasan Skala Database Monolitik

Ketika beban write dan volume data menembus batas komputasi satu mesin database.

SINGLE-INSTANCE I/O CEILING

Poin 1

Ukuran working set melampaui buffer pool RAM, memicu lonjakan disk I/O seek.

SINGLE-INSTANCE I/O CEILING

Poin 2

Write throughput tercekik oleh single WAL lock contention dan disk fsync queue.

SINGLE-INSTANCE I/O CEILING

Poin 3

Operasi maintenance tabel raksasa (ALTER TABLE, VACUUM) mengunci database berjam-jam.

MONOTONIC WRITE HOTSPOTS

Poin 1

Sharding key berbasis waktu (`created_at`) memusatkan 100% traffic tulis pada 1 shard aktif.

MONOTONIC WRITE HOTSPOTS

Poin 2

Shard lain menganggur sementara shard target kehabisan CPU dan disk IOPS.

MONOTONIC WRITE HOTSPOTS

Poin 3

Partisi fisik kehilangan fungsi load balancing akibat skew distribusi ekstrem.

SCATTER-GATHER EXHAUSTION

Poin 1

Query tanpa sharding key (`email`) memaksa broadcast paralel ke 100% shard cluster.

SCATTER-GATHER EXHAUSTION

Poin 2

Satu query lambat di satu shard menahan total latency (straggler problem).

SCATTER-GATHER EXHAUSTION

Poin 3

Throughput cluster ambruk dari $O(N)$ menjadi $O(1)$ di bawah konkurensi tinggi.

Dalam arsitektur backend skala besar, database monolitik pada akhirnya akan membentur batasan fisik perangkat keras. Ketika kapasitas buffer pool RAM terlewati, operasi baca-tulis terpaksa bergantung pada I/O disk yang lambat, sementara proses pemeliharaan tabel seperti VACUUM atau DDL migrations melumpuhkan throughput produksi. Lebih parah lagi, ketika tim mencoba melakukan sharding horizontal namun keliru memilih kunci partisi berbasis timestamp, seluruh beban tulis terkonsentrasi pada satu node aktif saja. Sementara itu, query pencarian sekunder tanpa shard key memicu banjir broadcast scatter-gather yang menghabiskan CPU di seluruh penjuru cluster.

Vertical scaling hits hardware walls; naive horizontal sharding without key topology introduces devastating write hotspots and broadcast storms.

Mengapa Hash Modulo Sederhana & Shard Monotonik Gagal?

Formula hash(key) % N memicu bencana migrasi 80% data saat menambah satu server.

```
func (m *ModuloRouter) GetShard(key string) (string, error) {
    // BAHAYA FATAL: Indeks berubah total saat N bertambah!
    h := fnv64a(key)
    idx := int(h % uint64(len(m.shards))) // Modulo N
    return m.shards[idx], nil
}
// Scale-out 4 -> 5 nodes: 79.84% data berpindah shard!
// Akibat: Network saturasi, cache invalidation masif, outage produksi.
```

Pendekatan naif kerap mengimplementasikan sharding menggunakan operator modulo sederhana hash(key) % N. Ketika sistem perlu menambah node ke-5 untuk mengatasi lonjakan beban, rumus modulo berubah seketika dan memaksa hampir 80 persen data berpindah antar server di jaringan. Proses resharding ini menyebabkan lonjakan transfer data terabyte, hilangnya cache memory, dan downtime sistemik. Selain itu, penggunaan auto-increment lokal di masing-masing shard mengakibatkan bentrokan primary key, sedangkan query tanpa shard key membebani seluruh server secara bersamaan.

KEGAGALAN FUNDAMENTAL TEKNIK NAIF

Data Shuffling Masif: Menambah 1 node dari N ke $N+1$ memindahkan $\approx \frac{N}{N+1}$ ($75\% - 80\%$) seluruh dataset.
Write Hotspot Monotonik: Shard key urut waktu (`2026-09-28`) mengarahkan 1.000 dari 1.000 insert ke satu node saja.
Scatter-Gather Stragglers: Mencari tanpa shard key membroadcast N node; latensi query sama dengan node terlambat (P_{99}).
Distributed ID Collision: Auto-increment native lokal menghasilkan ID duplikat lintas shard tanpa sequence coordinator.

Hash modulo forces global data relocation on resize; monotonic keys destroy horizontal write concurrency.

Mental Model: Logical Partitioning vs Physical Sharding

Membedakan partisi internal satu mesin dari kluster terdistribusi lintas node jaringan.

```
type PartitionRange struct {
    Name  string; Start time.Time; End time.Time // Range [Start, End)
}
type Table struct { // Single Engine Logical Partitioning
    Name string; partitions []*Partition // Pruning lokal via overlap check
}
type Cluster struct { // Multi-Node Physical Sharding
    shards map[string]*Shard // Server fisik independen
    router Router           // Consistent Hash Ring dengan Virtual Nodes
    gsi    *GlobalSecondaryIndex // Lookup Vindex mapping non-shard keys
}
```

Mental model sistem data terdistribusi bertumpu pada pemisahan konsep yang tegas antara partisi logis dan sharding fisik. Logical Partitioning adalah fitur mesin database tunggal (seperti declarative partitioning di PostgreSQL) yang membagi file fisik tabel berdasarkan rentang waktu untuk mempercepat query pruning dan pembuangan data historis tanpa beban garbage collection. Sebaliknya, Physical Sharding adalah arsitektur shared-nothing di mana setiap shard merupakan instance database mandiri dengan memori dan disk terisolasi. Sharding router dan Global Secondary Index (GSI) bertindak sebagai koordinator untuk mengarahkan trafik aplikasi secara akurat.

DIKOTOMI ARSITEKTUR FUNDAMENTAL

Logical Partitioning (1 Mesin): Membagi 1 tabel menjadi sub-tabel terpisah pada disk yang sama. CPU & RAM tetap shared.
Keunggulan Partisi: Query pruning mengeliminasi scan sub-tabel lain; `DROP PARTITION` instan menghapus data tanpa VACUUM.
Physical Sharding (N Mesin): Membagi dataset ke beberapa server mandiri (Shared-Nothing). Melampaui limit RAM/IOPS.
Router Ring & GSI: Router konsisten memetakan shard key ke node; GSI memetakan atribut sekunder (`email` -> `shardKey`).

Partitioning optimizes data lifecycle on one server; sharding scales compute and storage across independent physical nodes.

Pipeline End-to-End: Routing, Pruning, & GSI Resolution

Alur keputusan query: Evaluasi Shard Key → Consistent Hash Ring → Direct vs Scatter-Gather.

SEKUENSI EKSEKUSI DATABASE TERDISTRIBUSI

1. Client Query Ingestion: Request membawa payload record atau query kriteria pencarian.
2. Single-Node Path: Jika query temporal pada Table lokal → QueryRange() mengecek overlap [start, end) → Scan hanya 1 partisi (Pruned 3).
3. Sharded Write Path: Insert(rec) → Hash(rec.ShardKey) dievaluasi pada Consistent Hash Ring → Arahkan langsung ke Shard target.
4. Sharded Read with ShardKey: GetByShardKey(sk, id) → Router lookup $O(\log V)$ via binary search vnodes → 1 Direct Point Lookup.
5. Secondary Key Read (with GSI): GetByEmailUsingGSI(email, id) → GSI Lookup Vindex memetakan email → ShardKey → Direct Fetch (1 node).
6. Fallback Read (No ShardKey / No GSI): ScatterGatherBroadcast() → Goroutines paralel menembak seluruh N Shard → Gabungkan hasil.

Pipeline eksekusi ini menggambarkan bagaimana database menangani berbagai pola akses data secara efisien. Pada operasi temporal di single node, query planner langsung memangkas (pruning) partisi yang tidak relevan sehingga I/O disk berkurang drastis. Pada cluster sharded, operasi write dan read yang menyertakan sharding key diarahkan secara presisi ke satu node spesifik via consistent hash ring. Ketika client mencari data berdasarkan atribut non-shard key seperti email, Global Secondary Index memetakan email ke shard key terkait sehingga kueri tetap menjadi point-lookup 1-node dan menghindari bencana broadcast paralel.

Shard keys route directly; GSI resolves secondary lookups; scatter-gather remains the expensive last resort.

internal/sharding/sharding.go:280-340 • internal/partitioning/table.go:87-119

Implementasi Inti: Consistent Hashing Ring dengan Virtual Nodes

Algoritma routing ring internal/sharding/sharding.go: binary search vnode & FNV-1a hash.

```
func (ch *ConsistentHashRouter) GetShard(key string) (string, error) {
    ch.mu.RLock(); defer ch.mu.RUnlock()
    if len(ch.ring) == 0 { return "", ErrShardNotFound }
    h := hashKey(key)
    // Binary search vnode pertama dengan hash >= h
    idx := sort.Search(len(ch.ring), func(i int) bool {
        return ch.ring[i].hash >= h
    })
    if idx == len(ch.ring) { idx = 0 } // Wrap-around ke awal ring
    return ch.ring[idx].shardID, nil
}
```

Potongan kode ini merupakan mesin pengarah utama dalam modul internal/sharding/sharding.go. Struktur ConsistentHashRouter merepresentasikan ruang cincin 64-bit yang dipetakan oleh ribuan virtual nodes (misal 100 vnodes per physical shard). Saat kunci data masuk, fungsi hashKey menghitung nilai FNV-1a 64-bit, lalu fungsi sort.Search mengeksekusi binary search dalam waktu logaritmik O(log V) untuk menemukan virtual node pertama yang memiliki nilai hash lebih besar atau sama. Jika nilai hash melewati batas maksimum ring, pencarian wrap-around kembali ke indeks 0, menjamin distribusi seragam.

Binary search on a sorted virtual node ring delivers deterministic, uniform O(log V) routing.

Failure Path: Bencana Shard Key Monotonik & Resharding Modulo

Log terminal membuktikan 100% write hotspot pada 1 shard dan lonjakan 79.84% migrasi data.

TERMINAL HOTSPOT & RESHARDING TRACE

```
[Scenario A] Monotonic Key (date-string '2026-09-28') Sharding:
Node shard-0: 0 records [ ]
Node shard-1: 0 records [ ]
Node shard-2: 1000 records
[##############################] (100% HOTSPOT!)
Node shard-3: 0 records [ ]
Result: Severe Write Hotspot! 100% writes hit single shard.
------------------------------------------------------------------------
----------
[Scenario C] Resharding Scale-Out Comparison (4 Shards -> 5 Shards,
10,000 Keys):
Hash Modulo (N % M) Keys Remapped : 7,984 / 10,000 (79.84% moved!
BENCANA NETWORK)
Consistent Hashing Keys Remapped : 1,200 / 10,000 (12.00% moved!
MINIMAL SHUFFLE)
```

ANALISIS KEGAGALAN SISTEMATIS

Hotspot Monotonik: Shard key berbasis tanggal memusatkan 1.000 transaksi ke `shard-2`; 3 shard lainnya menganggur total.

Monotonic keys crush single instances with 100% write skew; modulo resizing triggers network-saturating data avalanches.

Mitigasi Lab: Consistent hashing ring membatasi relokasi data ke angka 12,00%, mendekati batas teoritis minimal $1/(N+1) = 20\%$.

Bukti failure path ini diambil langsung dari hasil eksekusi program demo cmd/demo/main.go. Pada Skenario A, penggunaan kunci sharding monotonik berbasis tanggal kalender menyebabkan 1.000 dari 1.000 data masuk ke satu node shard-2, membuktikan bagaimana shard key yang salah memusatkan beban 100 persen dan melumpuhkan kapasitas I/O satu server. Pada Skenario C, saat cluster melakukan scale-out dari 4 menjadi 5 node, algoritma hash modulo memindahkan 7.984 dari 10.000 key (79.84 persen). Sebaliknya, consistent hashing hanya memindahkan 1.200 key (12.00 persen), membuktikan keunggulan arsitekturalnya.

Mekanisme vs Gejala: Evaluasi Strategi Partisi & Sharding

Perbandingan mendalam: Karakteristik, skala beban, trade-off, dan ketahanan produksi.

Mekanisme	Arsitektur Target	Keunggulan Utama	Limitasi / Kerentanan Kritis	Ketahanan Produksi
Logical Partitioning	Single Engine (1 Node)	Query pruning, fast `DROP PARTITION`	Shared CPU/RAM; tidak menyelesaikan limit I/O fisik	Sangat Baik (Lifecycle Data)
Hash Modulo ($K \% N$)	Multi-Node Cluster	Algoritma $O(1)$ sangat sederhana	Relokasi data masif $\approx \frac{N}{N+1}$ saat resharding	Buruk (Dilarang saat Resizing Dinamis)
Consistent Hashing	Multi-Node Cluster	Relokasi minimal $\approx \frac{1}{N+1}$ saat scale-out	Membutuhkan virtual nodes untuk mencegah data skew	Unggul (Standar Industri)
Scatter-Gather Broadcast	Multi-Node Cluster	Mendukung query atribut non-shard key	Latensi P99 dibatasi node terlambat (straggler)	Rawan (Bottleneck pada Skala Besar)
Global Secondary	Multi-Node	lookup terarah 1–	Double-write overhead saat	(Untuk Query

Tabel perbandingan ini mengevaluasi spektrum lengkap solusi skalabilitas database. Logical partitioning sangat ampuh untuk mengelola siklus hidup data besar pada satu mesin (seperti membuang data lama secara instan tanpa operasi DELETE/VACUUM yang lambat), namun tidak mampu mengatasi keterbatasan memori dan CPU server tunggal. Untuk arsitektur multi-node, modulo hashing terbukti berbahaya saat cluster bertumbuh karena merombak mayoritas lokasi data. Consistent hashing dengan virtual nodes menjadi standar emas karena meminimalkan migrasi, sementara Global Secondary Index menghilangkan ketergantungan pada scatter-gather broadcast yang boros sumber daya.

Logical partitioning manages local table lifecycles; consistent hashing and GSI power scalable distributed clusters.

3 Keputusan Desain Kritis Arsitektur Sharding

Urutan keputusan wajib sebelum memecah database ke kluster multi-node terdistribusi.

1. SHARD KEY: HIGH CARDINALITY & UNIFORMITY

Pilih atribut dengan kardinalitas tinggi dan frekuensi rendah (contoh: `account_id`, `tenant_id`, `uuid`).
Haramkan nilai monotonik (`created_at`, auto-increment) sebagai range shard key untuk mencegah 100% write hotspot.
Terapkan Hashed Sharding jika beban kerja didominasi oleh operasi tulis sekuensial intensif.

2. ROUTER: CONSISTENT HASH RING & VNODES

Gunakan cincin hash konsisten (FNV-1a / Murmur3) dengan 50–150 virtual nodes per server fisik.
Virtual nodes mendistribusikan token secara acak-merata untuk mencegah ketimpangan beban (hot shard).
Saat scale-out dari N ke $N+1$, batasi perpindahan data hanya pada porsi $\approx 1/(N+1)$ dari total dataset.

3. SECONDARY ACCESS: GSI INDEXING POLICY

Identifikasi query non-shard key yang berfrekuensi tinggi (misal login via `email` pada cluster bertarget `user_id`).
Bangun Global Secondary Index (Lookup Vindex) untuk memetakan atribut
[illegible] trade-off double-write saat insert demi menjamin latensi baca tetap 1-
node point-lookup.

Tiga keputusan arsitektural ini menentukan keberhasilan transisi dari database monolitik ke sistem terdistribusi. Pertama, pemilihan sharding key harus memprioritaskan atribut dengan sebaran nilai yang sangat beragam (high cardinality) seperti tenant ID atau user ID guna mendistribusikan data secara seimbang ke seluruh node. Kedua, router wajib menerapkan consistent hashing dengan virtual nodes yang cukup (50 hingga 150 vnodes) agar penambahan node baru tidak memicu perombakan data global. Ketiga, query non-shard key harus dilayani oleh Global Secondary Index untuk mencegah broadcast paralel yang menghancurkan throughput cluster.

Select high-cardinality keys, govern routing with virtual node rings, and resolve secondary lookups via GSI tables.

Bukti Eksekusi: Verifikasi Pruning, Hashing, GSI, & UUIDv7

Output terminal go run ./cmd/demo membuktikan keunggulan pruning, distribusi vnodes, dan kecepatan GSI.

DEMO EXECUTION TRACE

```
=== LAB 32: DATABASE SHARDING AND PARTITIONING DEMONSTRATION ===
[1. Single-Node Partitioning] Query Range: 2026-04-01 to 2026-06-30
Records Found: 1 | Partitions Scanned: 1 / 4 (Pruned 3 partitions)
[2. Sharding Key Selection: Monotonic vs High-Cardinality]
Scenario A (date-string): Node shard-2 got 1000 records (100% Hotspot)
Scenario B (user_id) : Node shard-0: 400 | shard-2: 200 | shard-3: 400
(Distributed)
[3. Resharding Scale-out: 4 Shards -> 5 Shards (10,000 Keys)]
Hash Modulo (N % M) Keys Remapped : 7,984 / 10,000 (79.84% moved)
Consistent Hashing Keys Remapped : 1,200 / 10,000 (12.00% moved) |
Theoretical: ~20.00%
[4. Non-Sharded Attributes: Scatter-Gather vs GSI]
Scatter-Gather Query : Nodes Broadcasted: 4 / 4 | Execution Time: 92µs
Global Secondary Index: Nodes Broadcasted: 1 / 4 | Execution Time: 1µs (92×
Faster!)
[5. Distributed Unique ID Generation]
UUIDv7 (Time-Ordered 128-bit) : 01a0eb21-fc8c-7aba-abc5-bcb4361a5e7b
Sequence Block Allocator IDs : 1 2 3 4 5 6 7 8 (Block Size=5)
=== [DEMO COMPLETE] All database sharding & partitioning concepts
executed ===
```

Bukti eksekusi terminal membuktikan secara empiris seluruh klaim arsitektur dalam laboratorium ini. Pada modul partisi logis, query range pada kuartal kedua hanya memindai 1 partisi dan memangkas 3 partisi lainnya secara instan. Pada perbandingan resharding, consistent hashing ring membatasi relokasi data pada angka 12.00 persen dibandingkan modulo hashing yang merombak 79.84 persen data. Pada pengujian akses non-shard key, Global Secondary Index berhasil mengarahkan query langsung ke 1 shard dalam waktu 1 mikrodetik, mengungguli metode scatter-gather yang membroadcast 4 shard dalam 92 mikrodetik.

Pruning eliminates 75% partition I/O; consistent hashing cuts migration 6.6×; GSI delivers 92× faster point lookups.

cmd/demo/main.go:1-210 • engineering/03-execution-result.md:65-111

YANG TIDAK DIBUKTIKAN

Batasan ruang lingkup laboratorium yang wajib dipahami sebelum implementasi produksi.

YANG TIDAK DIBUKTIKAN

Penyimpanan tabel dan shard berbasis in-memory Go struct; tidak menguji disk persistence engine (InnoDB, RocksDB, LSM-Tree).
Tidak menyertakan Two-Phase Commit (2PC) coordinator untuk transaksi cross-shard atomik (XA transaction).
Global Secondary Index (GSI) dimodelkan secara in-memory sinkron; tidak menguji asynchronous eventual consistency replikasi GSI.
Tidak menyertakan dynamic shard split / auto-rebalancing engine otomatis saat kapasitas fisik storage mencapai batas.
Tidak menyimulasikan network partitions, node failures, atau consensus replication (Raft/Paxos) antar replika shard.
Tidak menguji distributed deadlock detection saat dua transaksi cross-shard saling mengunci record di shard berlawanan.

Laboratorium ini dirancang mandiri untuk membuktikan mekanisme algoritma routing, partisi logis, dan mitigasi query tanpa dependensi eksternal. Namun demikian, implementasi ini memiliki batasan ruang lingkup tegas yang harus diperhatikan oleh arsitek sistem. Komponen shard dan GSI berjalan di dalam memori proses tunggal tanpa persistensi disk fisik. Lab ini tidak menguji protokol Two-Phase Commit (2PC) terdistribusi yang diperlukan untuk transaksi atomik lintas shard, dan tidak memodelkan replikasi konsensus Raft atau deteksi deadlock terdistribusi yang umum terjadi pada database skala enterprise.

Acknowledge educational bounds: in-memory routing models routing algorithms; production requires 2PC engines and consensus replication.

Trade-off Arsitektur: Sharding vs Kompleksitas Terdistribusi

Menilai keuntungan kapasitas horisontal terhadap beban biaya operasional dan konsistensi data.

KEUNTUNGAN & KAPASITAS SKALABILITAS

Skalabilitas Horisontal Linear: Menembus batasan memori RAM, disk IOPS, dan write bandwidth mesin database tunggal.
Blast Radius Terisolasi: Kerusakan hardware pada Shard 2 tidak mematikan ketersediaan data pada Shard 0, 1, dan 3.
Query Pruning Efisien: Rentang partisi logis memangkas scan disk secara drastis untuk tabel berbasis waktu.

BIAYA OPERASIONAL & KOMPLEKSITAS

Kehilangan JOIN Relasional: Operasi SQL JOIN lintas shard key mustahil dilakukan tanpa scatter-gather atau agregasi di aplikasi.
Overhead Transaksi Terdistribusi: Two-Phase Commit (2PC) melipatgandakan latensi jaringan dan berisiko fractured reads.
Double-Write Overhead pada GSI: Menjaga indeks sekunder global membutuhkan write ganda yang memperlambat operasi insert.

Keputusan mengadopsi physical sharding selalu membawa kompromi teknik yang signifikan. Di satu sisi, sistem memperoleh skalabilitas linear dan isolasi kegagalan di mana matinya satu server tidak melumpuhkan seluruh aplikasi. Namun di sisi lain, pengembang harus membayar harga arsitektural yang mahal: hilangnya kapabilitas SQL JOIN antar tabel yang berbeda shard, tingginya latensi koordinasi transaksi Two-Phase Commit, serta keharusan mengelola overhead double-write untuk menjaga konsistensi Global Secondary Index.

Sharding buys unbounded scale and fault isolation at the cost of cross-shard joins, 2PC latency, and index dual-writes.

Validasi Konkurensi: 200 Worker Simultan pada Sharded Cluster

TestConcurrentClusterAccess membuktikan integritas state di bawah konkurensi Insert, Read, dan GSI.

```
func TestConcurrentClusterAccess(t *testing.T) {
    router := sharding.NewConsistentHashRouter(50, []string{"s1", "s2", "s3", "s4"})
    cluster := sharding.NewCluster(router)
    var wg sync.WaitGroup; numOps := 200
    for i := 0; i < numOps; i++ {
        wg.Add(1)
        go func(idx int) {
            defer wg.Done()
            sk := fmt.Sprintf("tenant-%d", idx%10)
            recID := fmt.Sprintf("id-%d", idx)
            email := fmt.Sprintf("user-%d@test.com", idx)
            _ = cluster.Insert(sharding.Record{ID: recID, ShardKey: sk, Email: email, Data: "concurrent-test"})
            _, _ = cluster.GetByShardKey(sk, recID)
            _, _ = cluster.GetByEmailUsingGSI(email, recID)
        }(i)
    }
    wg.Wait()
    // Assert: total seluruh record di seluruh shard == 200 (Zero Data Loss)
}
```

Uji konkurensi TestConcurrentClusterAccess memverifikasi ketahanan cluster saat diserbu oleh 200 goroutine simultan yang melakukan insert, pembacaan berbasis shard key, dan point-lookup via GSI. Karena setiap instance Shard dan struktur GlobalSecondaryIndex memiliki sync.RWMutex masing-masing, penguncian data bersifat granular sehingga operasi pada shard-1 tidak menghalangi operasi pada shard-2. Pengujian dengan flag -race membuktikan ketiadaan memory race condition dan menegaskan integritas data tersimpan secara utuh tepat 200 record.

VERIFIKASI THREAD SAFETY & RACE DETECTOR

RWMutex Granular: Setiap Shard dan GSI dilindungi oleh `sync.RWMutex` independen, mencegah lock contention antar shard.
Zero Data Race: Eksekusi `go test -race` lulus 100% bersih tanpa mendeteksi data race condition pada 200 goroutine.
Total Invariant: 200 transaksi insert terdistribusi secara atomik, verifikasi cluster count membuktikan tidak ada data hilang.

Granular per-shard mutexes isolate concurrency contention; race detectors verify thread-safe distributed routing.

labs/32-database-sharding-and-partitioning/tests/sharding_test.go:212-249

Before vs After: Evolusi Arsitektur Manajemen Data

Transformasi dari database monolitik rentan bottleneck menuju cluster sharded terkoordinasi.

BEFORE (MONOLITHIC DB & NAIVE MODULO)

Kapasitas: Terkunci pada limit hardware 1 server fisik; disk I/O mentok saat data membesar.
Query Temporal: Full table scan membebani jutaan row; maintenance `DELETE` memicu table bloat.
Resharding: Menggunakan rumus hash modulo; menambah node baru memindahkan 80% data.
Query Sekunder: Menembak seluruh database tanpa filter; lonjakan CPU membakar server.
Primary Keys: Auto-increment lokal bertabrakan jika dipisah ke beberapa node.

AFTER (PARTITIONING, CONSISTENT RING, & GSI)

Kapasitas: Skalabilitas horisontal multi-node terdistribusi; beban write terbagi merata.
Query Temporal: Range logical partitioning memangkas 75% partisi (query pruning); drop instan.
Resharding: Consistent hashing ring dengan vnodes membatasi migrasi hanya $\approx 12\% - 16\%$.
Query Sekunder: Global Secondary Index memetakan atribut sekunder unik terurut waktu.

Perbandingan before-after ini merangkum evolusi menyeluruh dari arsitektur penyimpanan data. Pada paradigma lama, tim bergantung pada satu server monolitik besar yang rentan mengalami disk bottleneck, menggunakan rumus modulo rapuh yang memicu badai migrasi data saat scaling, serta menderita bentrokan primary key. Dengan arsitektur modern, tabel lokal dipangkas secara logis untuk efisiensi siklus hidup data, cluster didistribusikan via consistent hashing ring dengan virtual nodes, pencarian sekunder diarahkan via GSI, dan ID unik digenerasi secara aman via standar UUIDv7.

Before: hardware ceilings, 80% resharding churn, and broadcast storms. After: 75% pruning, 12% scale-out shuffle, and 1-node GSI routing.

Test 1: Validasi Query Pruning pada Range Partitioning

TestPartitionPruning membuktikan hanya 1 dari 3 partisi yang discan dan drop tabel instan.

```
func TestPartitionPruning(t *testing.T) {
    ranges := []partitioning.PartitionRange{
        {Name: "p2026_01", Start: time.Date(2026, 1, 1, 0, 0, 0, 0, time.UTC), End: time.Date(2026, 2, 1, 0, 0, 0, 0, time.UTC)},
        {Name: "p2026_02", Start: time.Date(2026, 2, 1, 0, 0, 0, 0, time.UTC), End: time.Date(2026, 3, 1, 0, 0, 0, 0, time.UTC)},
        {Name: "p2026_03", Start: time.Date(2026, 3, 1, 0, 0, 0, 0, time.UTC), End: time.Date(2026, 4, 1, 0, 0, 0, 0, time.UTC)},
    }
    tbl := partitioning.NewTable("orders", ranges)
    // Insert data Jan dan Feb...
    res := tbl.QueryRange(febStart, febEnd)
    // Assert: res.PartitionsScanned == 1 && res.TotalPartitions == 3 (Pruning Sukses!)
    // Drop partition Jan p2026_01...
    // Assert: tbl.PartitionCount() == 2 (Hapus instan tanpa scan tabel)}
```

VERIFIKASI INVARIAN QUERY PRUNING

Overlap Condition Pruning: Logika `start.Before(p.Range.End) && end.After(p.Range.Start)` mengabaikan partisi Jan dan Mar.
Eliminasi Disk I/O: Query hanya membaca memori partisi Feb, menghemat 66.7% operasi I/O pemindaian.
Fast Drop Partition: Menghapus partisi `p2026_01` membuang seluruh row di dalamnya secara instan tanpa VACUUM bloat.

Unit test pertama memvalidasi mekanisme pemangkasan partisi pada internal/partitioning/table.go. Tabel orders dikonfigurasikan dengan tiga rentang partisi bulanan. Saat kueri range dieksekusi khusus untuk bulan Februari 2026, sistem secara cerdas mengevaluasi batasan waktu partisi dan hanya memindai partisi p2026_02 (PartitionsScanned=1 dari 3). Selain itu, pengujian membuktikan bahwa method DropPartition menghapus seluruh data bulan Januari seketika secara $O(1)$ tanpa perlu memindai dan menghapus baris satu per

Pruning logic start < end && end > start bypasses irrelevant partitions; dropping partitions avoids VACUUM overhead.

Test 2: Evaluasi Relokasi Data Scale-Out (5.000 Keys)

TestRoutingAndConsistentHashRelocation membuktikan Modulo merelokasi 75.12% vs Consistent Hash 16.00%.

```
func TestRoutingAndConsistentHashRelocation(t *testing.T) {
    numKeys := 5000
    // 1. Modulo router test: 3 nodes -> 4 nodes
    modRouter := sharding.NewModuloRouter([]string{"shard-1", "shard-2", "shard-3"})
    // Catat penempatan awal...
    modRouter.AddShard("shard-4") // Scale-out 3 -> 4
    // Hash Modulo memindahkan 3756 / 5000 keys (75.12%)
    // 2. Consistent Hash router test (150 vnodes): 3 nodes -> 4 nodes
    chRouter := sharding.NewConsistentHashRouter(150, []string{"shard-1", "shard-2", "shard-3"})
    // Catat penempatan awal...
    chRouter.AddShard("shard-4") // Scale-out 3 -> 4
    // Consistent Hash memindahkan 800 / 5000 keys (16.00%)
}
```

Pengujian kedua membandingkan dampak scale-out cluster dari 3 node menjadi 4 node terhadap 5.000 kunci data unik. Algoritma modulo hashing terbukti merelokasi 3.756 kunci (75.12 persen), memvalidasi model matematika di mana penambahan node ke-4 mengacak ulang tiga perempat penempatan data. Sebaliknya, router consistent hashing dengan 150 virtual nodes hanya memindahkan 800 kunci (16.00 persen). Data yang tersisa (84 persen) tetap berada di shard aslinya tanpa perlu dipindahkan sama sekali melalui jaringan.

ANALISIS RASIO PERPINDAHAN KUNCI

Modulo Churn: Formula $K \pmod N$ merelokasi 75.12% kunci (mendekati prediksi teoritis $3/4 = 75.00\%$).
Consistent Hash Stability: Cincin hash dengan 150 vnodes hanya merelokasi 16.00% kunci (dalam batas aman $5\% - 40\%$).
Efisiensi Jaringan: Consistent hashing menghemat 78.7% beban transfer data antar server saat cluster melakukan ekspansi kapasitas.

Modulo reshuffling displaces 75% of keys; consistent hashing confines key migration to the newly added sector.

Test 3: Eliminasi Broadcast via Global Secondary Index

TestClusterScatterGatherAndGSI membuktikan point-lookup 1-node dan timeout cancellation aman.

```
// 1. Direct Point-Lookup menggunakan ShardKey
rec, err := cluster.GetByShardKey("tenant-A", "rec-1") // Langsung ke Shard 0
// 2. Lookup via Global Secondary Index (GSI)
recGSI, err := cluster.GetByEmailUsingGSI("bob@example.com", "rec-2")
// Assert: recGSI.Data == "Bob Data" (Tepat 1 shard dihubungi, 0 broadcast overhead)
// 3. Fallback Scatter-Gather Broadcast
sgRes := cluster.ScatterGatherBroadcast(func(r sharding.Record) bool { return r.Email == "carol@example.com" })
// Assert: sgRes.ShardsQueried == 3 (Broadcast menembus seluruh 3 shard)
// 4. Scatter-Gather dengan Context Timeout / Cancellation
ctx, cancel := context.WithCancel(context.Background()); cancel()
canceledRes := cluster.ScatterGatherBroadcastWithContext(ctx, predicate)
// Assert: canceledRes.ShardResponded == 0 (Goroutine dibatalkan seketika)
```

CHECKPOINT OPTIMASI NON-SHARDED QUERY

GSI Direct Routing: Memetakan `email -> shardKey` memungkinkan pemanggilan langsung tanpa menyentuh shard lain.
Scatter-Gather Broadcast: Menanyakan 3/3 shard secara paralel via Goroutines; mahal namun menjamin kelengkapan data.
Context Safety: Channel cancellation `ctx.Done()` menghentikan pemrosesan goroutine di semua shard jika client timeout.

Unit test ketiga menguji arsitektur pencarian atribut sekunder yang tidak dijadikan sharding key. Fungsi GetByEmailUsingGSI membuktikan bahwa pemetaan Global Secondary Index memungkinkan sistem menemukan data hanya dengan mengakses 1 shard spesifik tanpa membroadcast ke shard lain. Ketika pencarian dilakukan tanpa bantuan GSI melalui ScatterGatherBroadcast, seluruh 3 shard terpaksa dipindai secara paralel. Pengujian juga memvalidasi bahwa jika klien membatalkan request melalui context.Context, seluruh goroutine broadcast langsung berhenti bekerja dengan aman.

GSI transforms cluster-wide broadcast queries into targeted single-node reads; context cancels rogue scatter goroutines.

Studi Kasus Industri: Vitess (YouTube) & Citus (Postgres)

Bagaimana sistem skala internet memecahkan masalah sharding, routing, dan cross-shard queries.

VITESS: YOUTUBE & SLACK HORIZONTAL SCALING

Poin 1

Vindex Routing Engine: Vitess menggunakan abstraksi Vindex (Functional & Lookup Vindexes) untuk memetakan SQL query ke tablet MySQL.

VITESS: YOUTUBE & SLACK HORIZONTAL SCALING

Poin 2

Lookup Vindex as GSI: Indeks sekunder global disimpan di keyspace terpisah untuk melayani query non-shard key (`user_email`).

VITESS: YOUTUBE & SLACK HORIZONTAL SCALING

Poin 3

Sequence Keyspace: Menghindari bentrokan ID numerik dengan mengalokasikan blok auto-increment secara terpusat.

CITUS: DISTRIBUTED POSTGRESQL EXTENSION

Poin 1

Co-Location Strategy: Tabel yang memiliki relasi (misal `orders` dan `order_items`) di-shard menggunakan `tenant_id` yang sama.

CITUS: DISTRIBUTED POSTGRESQL EXTENSION

Poin 2

Local Foreign Key Joins: Data terkait berada pada physical worker yang sama, memungkinkan query SQL JOIN dieksekusi lokal.

CITUS: DISTRIBUTED POSTGRESQL EXTENSION

Poin 3

Distributed Query Planner: Mentransformasikan SQL kompleks menjadi paralel query snippets ke masing-masing node worker.

Arsitektur industri membuktikan bahwa prinsip-prinsip dalam lab ini adalah fondasi dari platform data terdistribusi terkemuka. Vitess yang digunakan oleh YouTube dan Slack memelopori konsep Vindexes di mana router cerdas VTGate memetakan query ke ratusan shard MySQL tanpa mengubah kode aplikasi. Sementara itu, Citus pada ekosistem PostgreSQL menekankan strategi Co-Location: dengan menyelaraskan sharding key antara tabel induk dan anak, transaksi dan relasi JOIN dapat diselesaikan sepenuhnya di level server lokal tanpa melintasi jaringan.

Co-locate related tables by tenant ID to preserve local SQL joins; rely on Vindex engines for transparent routing.

Analisis Teknis: Solusi Primary Key Terdistribusi

Mencegah tabrakan primary key: RFC 9562 UUIDv7 vs Sequence Block Allocation (Vitess).

RFC 9562 UUIDV7 (TIME-ORDERED 128-BIT)

Poin 1

Struktur Bit: 48-bit Unix timestamp ms | 4-bit Ver (0111) | 12-bit rand_a | 2-bit Var (10) | 62-bit rand_b.

RFC 9562 UUIDV7 (TIME-ORDERED 128-BIT)

Poin 2

Keunggulan B-Tree: Terurut secara temporal ($u_1 < u_2$); mencegah page splitting masif pada indeks database.

RFC 9562 UUIDV7 (TIME-ORDERED 128-BIT)

Poin 3

Zero Coordination: Tidak memerlukan koordinasi jaringan, server sentral, atau lock terpusat antar worker.

SEQUENCE BLOCK ALLOCATION (VITESS-STYLE)

Poin 1

Mekanisme Blok: Worker meminta rentang blok ID (misal ukuran blok = 1.000 ID) dari central sequence counter.

SEQUENCE BLOCK ALLOCATION (VITESS-STYLE)

Poin 2

Eksekusi Lokal: 1.000 operasi insert berikutnya mengalokasikan ID sekuensial di memori lokal tanpa panggilan jaringan.

SEQUENCE BLOCK ALLOCATION (VITESS-STYLE)

Poin 3

Efisiensi Jaringan: Mengurangi 99.9% round-trip database dibandingkan query auto-increment sekuensial tradisional.

Dalam database terdistribusi, fitur auto-increment bawaan satu mesin tidak dapat digunakan karena akan menimbulkan tabrakan primary key antar shard. Lab ini menguji dua solusi standar industri. Pertama, RFC 9562 UUIDv7 menghasilkan ID 128-bit independen yang menyematkan 48-bit timestamp pada bit awal, menjamin sifat time-ordered yang ramah indeks B-tree tanpa koordinasi jaringan. Kedua, Sequence Block Allocation meminta jatah alokasi blok angka (misal blok 1.000 ID) ke server sentral, memungkinkan worker menerbitkan ID integer berurutan secara cepat di memori lokal.

UUIDv7 provides zero-coordination B-tree friendly IDs; sequence block allocators preserve sequential integers efficiently.

Checklist Implementasi Produksi Sharding & Partitioning

9 poin verifikasi arsitektur sebelum menggelar cluster database terdistribusi ke produksi.

- ✓ [ ] Mulai dari Logical Table Partitioning pada 1 mesin sebelum memutuskan sharding multi-node.
- ✓ [ ] Evaluasi Shard Key: Wajib berkardinalitas tinggi (tenant_id / user_id) dan tidak menggunakan nilai monotonik.
- ✓ [ ] Terapkan Hashed Sharding jika beban kerja didominasi oleh operasi tulis temporal intensif.
- ✓ [ ] Gunakan Consistent Hashing dengan 50–150 virtual nodes per server fisik untuk stabilitas scale-out.
- ✓ [ ] Siapkan Global Secondary Index (Lookup Vindex) untuk query non-shard key yang berfrekuensi tinggi.
- ✓ [ ] Terapkan timeout dan context cancellation ketat pada seluruh query fallback Scatter-Gather broadcast.
- ✓ [ ] Gunakan RFC 9562 UUIDv7 atau Sequence Block Allocation untuk primary key terdistribusi tanpa tabrakan.
- ✓ [ ] Kelompokkan tabel berelasi dalam satu shard key yang sama (Co-Location) demi menjaga integritas JOIN lokal.

Maximize single-node partitioning first; when sharding, mandate high-cardinality keys, vnode rings, and table co-location.

- ✓ [ ] Rancang arsitektur tanpa ketergantungan pada transaksi Two-Phase Commit (2PC) cross-shard berkepanjangan.

Anti-Pattern: Kesalahan Desain Sharding & Reproduksi Lab

Kekeliruan arsitektur yang melumpuhkan cluster dan cara membuktikannya langsung di lab.

ANTI-PATTERN DISTRIBUTED DATA UMUM

Monotonic Shard Key: Menggunakan tanggal atau auto-increment sebagai shard key, memicu 100% write hotspot.
Modulo Hash Resharding: Menggunakan rumus `key % N` yang memaksa 80% data berpindah saat penambahan node.
Unindexed Scatter-Gather: Membiarkan query pencarian sekunder melakukan broadcast 100% node di setiap request.
Distributed Foreign Keys: Mencoba menegakkan relasi JOIN antar shard berbeda tanpa co-location data.

CARA REPRODUKSI KEGAGALAN DI LAB

1. Eksekusi `demoShardingHotspots()` di `cmd/demo/main.go`: Saksikan 1.000 record menumpuk 100% di `shard-2`.
2. Jalankan `TestRoutingAndConsistentHashRelocation`: Saksikan Hash Modulo memindahkan 75.12% key pada scale-out.
3. Bandingkan latensi Scatter-Gather (92µs) vs GSI (1µs) di `demoScatterGatherVsGSI()` untuk membuktikan broadcast penalty.

Mengabaikan prinsip dasar partisi database dapat melumpuhkan seluruh infrastruktur backend. Anti-pattern paling umum adalah memilih sharding key monotonik seperti tanggal transaksi yang memusatkan seluruh write traffic pada satu mesin database yang malang. Kesalahan fatal lainnya adalah menggunakan rumus hash modulo sederhana yang memicu banjir data saat menambah kapasitas server. Pengembang dapat mereproduksi skenario kegagalan ini secara langsung di lab melalui fungsi demo dan unit test yang telah disediakan.

Never shard on monotonic keys; never scale out using raw modulo hashing; verify failure thresholds through lab tests.

Verifikasi Mandiri: Uji & Jalankan Lab di Mesin Anda

Reproduksi seluruh unit test, deteksi data race, dan simulasi cluster sharded di terminal Anda.

```
cd labs/32-database-sharding-and-partitioning
go test -v -count=1 ./...
go test -race -v -count=1 ./...
go run ./cmd/demo
```

RINGKASAN METRIK LABORATORIUM

Test Suite: 5/5 test suites lulus 100% (Pruning, Resharding Relocation, Scatter-Gather & GSI, UUIDv7, Concurrency).
Race Detector: Bersih mutlak dari data race condition pada 200 goroutine simultan (1.405s pass).
Zero Dependencies: Murni Go Standard Library (`sync`, `hash/fnv`, `crypto/rand`, `sort`, `time`) tanpa dependensi eksternal.

Laboratorium ini dirancang untuk dapat diverifikasi secara mandiri dalam hitungan detik tanpa memerlukan instalasi database eksternal. Seluruh logika sharding router, virtual nodes, partition pruning, dan generator UUIDv7 dibangun murni menggunakan pustaka standar Go. Perintah go test -v mengeksekusi pengujian komprehensif mulai dari verifikasi query pruning hingga rasio relokasi consistent hashing. Perintah go test -race menjamin thread-safety di bawah konkurensi tinggi, dan go run demo memperlihatkan perbandingan performa secara visual.

Zero-dependency standard library code enables deterministic, transparent verification of distributed data architectures.

Prinsip Arsitektur: Desain Database Skala Besar

Tiga hukum fundamental dalam merancang penyimpanan data yang tahan uji skala waktu.

1. PARTISI LOGIS SEBELUM SHARDING FISIK

Gunakan declarative table partitioning pada satu mesin untuk query pruning dan manajemen siklus hidup data.
Tunda sharding fisik terdistribusi sampai batasan hardware vertikal (I/O, RAM, Write lock) benar-benar terlampaui.

2. SHARD KEY MENENTUKAN STRUKTUR SISTEM

Pilih sharding key berkardinalitas tinggi yang menyelaraskan pola akses baca dan tulis aplikasi.
Co-locate entitas data terkait di bawah satu kunci partisi guna menghindari transaksi terdistribusi Two-Phase Commit.

3. MINIMALKAN PERPINDAHAN & BROADCAST

Gunakan Consistent Hashing dengan virtual nodes agar ekspansi cluster hanya memindahkan porsi data minimal $O(1/N)$.
Lindungi sistem dari broadcast scatter-gather menggunakan Global Secondary Indexes untuk akses atribut sekunder.

Tiga prinsip arsitektural ini merupakan pedoman abadi bagi arsitek perangkat lunak dalam merancang sistem penyimpanan data. Pertama, jangan terburu-buru melakukan sharding terdistribusi; maksimalkan partisi logis dan optimasi query pada mesin tunggal sebelum menambah kompleksitas jaringan. Kedua, pahami bahwa sharding key adalah keputusan topologi yang menentukan struktur sistem: co-locate data terkait untuk mempertahankan transaksi dan relasi lokal. Ketiga, rancang arsitektur routing yang meminimalkan pergerakan data saat scaling dan mengeliminasi ketergantungan pada query broadcast.

Exhaust single-node partitioning first; co-locate data by high-cardinality keys; govern scale-out with consistent hash rings.

Partisi Cerdas, Sharding Terukur, Skalabilitas Tanpa Batas.

Logical partitioning prunes locally; consistent hashing scales horizontally; GSI eliminates broadcast overhead.

Software Engineering Lab 32 - Database Sharding & Table Partitioning

internal/partitioning/table.go — Single-node range-based logical partitioning with query pruning & fast drop
internal/sharding/sharding.go — ModuloRouter vs ConsistentHashRouter (with vnodes), Cluster, & Scatter-Gather
internal/idgen/idgen.go — RFC 9562 UUIDv7 (128-bit time-ordered) & Vitess-style Sequence Block Allocator
tests/sharding_test.go — 5 comprehensive test suites covering pruning, resharding, GSI, IDs, & concurrency
cmd/demo/main.go — Interactive benchmarks demonstrating hotspots, resharding churn, and 92× GSI speedup

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/32-database-sharding-and-partitioning`

Lukman / lukman-ss

internal/partitioning/table.go — Single-node range-based logical partitioning with query pruning & fast drop
internal/sharding/sharding.go — ModuloRouter vs ConsistentHashRouter (with vnodes), Cluster, & Scatter-Gather
internal/idgen/idgen.go — RFC 9562 UUIDv7 (128-bit time-ordered) & Vitess-style Sequence Block Allocator
tests/sharding_test.go — 5 comprehensive test suites covering pruning, resharding, GSI, IDs, & concurrency
cmd/demo/main.go — Interactive benchmarks demonstrating hotspots, resharding churn, and 92× GSI speedup
Commit SHA: 04c8c8a4
Unit Tests: 5/5 PASS (100% clean, 0.446s)
Race Detector: PASS (zero data race, 1.405s)
Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/32-database-sharding-and-partitioning